Tóm Tắt Chương Trình Giải Bài Toán Pickup-Delivery Problem với Drone Resupply (GA + Tabu Search)

1 Tổng Quan Bài Toán

Chương trình giải bài toán **Pickup-Delivery Problem with Drone Resupply (PDP-DR)**, trong đó một đội xe tải (trucks) và drone phối hợp phục vụ khách hàng. Mục tiêu là **tối thiểu hóa makespan** $C_{\max}$ — thời gian hoàn thành sớm nhất khi tất cả khách hàng đều được phục vụ.

1.1 Phân Loại Khách Hàng

Có hai loại khách hàng:

- **C1 (Type D)**: Khách hàng nhận hàng qua *drone resupply*. Hàng hóa xuất hiện tại depot vào thời điểm r_i (ready time). Drone mang hàng từ depot bay đến *điểm hẹn* (rendezvous point) để giao cho truck, sau đó truck đi giao cho khách.
- **C2 (Type P → DL)**: Khách hàng pickup-delivery truyền thống. Truck đến điểm Pickup (P) lấy hàng, rồi đến điểm Delivery (DL) giao hàng. Ràng buộc: P phải được phục vụ **trước** DL cùng cặp.

1.2 Mô Hình One Rendezvous

Drone hoạt động theo mô hình **One Rendezvous**:

1. Drone xuất phát từ **depot**, mang tối đa Q_d gói hàng.
2. Drone bay **thẳng** đến **một điểm hẹn duy nhất** (điểm hẹn = vị trí khách hàng đầu tiên trong trip).
3. Tại điểm hẹn, drone giao **tất cả** gói hàng cho truck.
4. Drone **quay về depot** ngay lập tức.
5. Truck tự đi giao hàng cho từng khách hàng trong trip.

1.3 Hàm Mục Tiêu

$$\min\ C_{\max} = \max\left\{\max_{k\in\mathcal{K}} T_k^{\text{end}},\ \ \max_{d\in\mathcal{D}} T_d^{\text{return}}\right\} \tag{1}$$

trong đó:

- T_k^{end}: thời gian truck k phục vụ xong khách hàng cuối cùng
- T_d^{return}: thời gian drone d quay về depot

- $\mathcal{K}$: tập xe tải, $\mathcal{D}$: tập drone

2 Tham Số Bài Toán

Bảng 1: Tham số mặc định của bài toán

Tham số	**Ý nghĩa**	**Giá trị**
$\lvert\mathcal{K}\rvert$	Số xe tải	2
$\lvert\mathcal{D}\rvert$	Số drone	2
v_{truck}	Tốc độ xe tải	30 km/h
v_{drone}	Tốc độ drone	60 km/h
Q_k	Tải trọng xe tải	50
Q_d	Tải trọng drone (per trip)	$\begin{cases} 2 & n \leq 20 \\ 10 & n > 20 \end{cases}$
s_{truck}	Thời gian phục vụ tại mỗi điểm	3 phút
E_d	Endurance (thời gian bay tối đa)	90 phút
t_{resupply}	Thời gian resupply (drone $\rightarrow$ truck)	5 phút
t_{load}	Thời gian load hàng tại depot	5 phút
t_{receive}	Thời gian truck nhận hàng tại depot	5 phút
Khoảng cách		
Truck	Manhattan: $d_M(i,j) = \lvert x_i - x_j\rvert + \lvert y_i - y_j\rvert$	
Drone	Euclidean: $d_E(i,j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$	

3 Mã Hóa Lời Giải (Encoding)

3.1 Chromosome (Sequence Encoding)

Mỗi cá thể trong quần thể được biểu diễn bằng một **hoán vị** (permutation) của n khách hàng:

$$\pi = (\pi_1, \pi_2, \ldots, \pi_n), \quad \pi_i \in \{1, 2, \ldots, n\} \tag{2}$$

Chromosome **không chứa separator** — chỉ là thứ tự phục vụ khách hàng. Việc gán khách hàng cho xe tải nào được thực hiện bởi hàm `decodeAndEvaluate` theo chiến lược tham lam.

3.2 Assignment Encoding (cho Local Search)

Ngoài sequence, mỗi lời giải còn có **assignment encoding** gồm 3 vector:

- **truck_assign**$[i] \in \{0, 1, \ldots, \lvert\mathcal{K}\rvert - 1\}$: Xe tải phục vụ khách hàng π_i
- **drone_assign**$[i] \in \{0, 1, \ldots, \lvert\mathcal{D}\rvert\}$: Drone phục vụ ($0$ = depot return, > 0 = drone ID)
- **break_bit**$[i] \in \{0, 1\}$: 1 = bắt đầu drone trip mới, 0 = tiếp tục trip hiện tại

4 Kiến Trúc Chương Trình

Bảng 2: Cấu trúc file mã nguồn

File	Chức năng
`main_ga_tabu.cpp`	Điểm vào chính (entry point), cấu hình tham số, gọi GA+Tabu
`pdp_types.h`	Cấu trúc dữ liệu: `PDPData`, `PDPSolution`, `ResupplyEvent`, `TruckRouteInfo`
`pdp_reader.cpp`	Đọc file instance, xây dựng ma trận khoảng cách Manhattan & Euclidean
`pdp_init.cpp`	Khởi tạo quần thể ban đầu (Random, Greedy Time, Sweep, Nearest Neighbor)
`pdp_fitness.cpp`	Hàm fitness (`decodeAndEvaluate`), drone trip evaluation, merge strategies, Assignment LS
`pdp_ga.cpp`	Genetic Algorithm: crossover, mutation, selection, adaptive operators
`pdp_tabu.cpp`	Tabu Search: 6 loại move, adaptive weights, aspiration criterion
`pdp_localsearch.cpp`	Integrated Local Search: truck operators + drone operators + SA acceptance
`pdp_validation.cpp`	Kiểm tra tính hợp lệ của lời giải
`pdp_utils.cpp`	Các hàm tiện ích (Euclidean distance, in lời giải, ...)

5 Khởi Tạo Quần Thể

Quần thể ban đầu gồm $N = 200$ cá thể, được khởi tạo bằng 4 phương pháp:

Bảng 3: Phân bổ khởi tạo quần thể

Phương pháp	Tỷ lệ	Mô tả
Random	10%	Hoán vị ngẫu nhiên các khách hàng
Greedy Time	30%	Tham lam theo ready_time + travel_time, chọn ngẫu nhiên trong top 5
Sweep	30%	Sắp xếp theo góc cực (polar angle) từ depot, góc gốc ngẫu nhiên
Nearest Neighbor	30%	Tham lam theo khoảng cách gần nhất, điểm bắt đầu ngẫu nhiên

Tất cả cá thể sau khi tạo đều được **repair** để đảm bảo:

- Mỗi khách hàng xuất hiện đúng 1 lần
- Ràng buộc P trước DL (precedence constraint) được thỏa mãn

6 Hàm Fitness: `decodeAndEvaluate`

Đây là hàm cốt lõi, giải mã chromosome thành lời giải hoàn chỉnh.

Algorithm 1 decodeAndEvaluate(π, data)

```
Khởi tạo |K| xe tải tại depot, |D| drone tại depot
for i = 1 to n do
    v ← π_i {Khách hàng tiếp theo trong sequence}
    if v là DL (delivery) then
        Gán v cho xe đã pickup P tương ứng (ràng buộc precedence)
    else
        Chọn xe tải k* = arg min_k T_k^available {Xe rảnh sớm nhất}
        if v là Type D (C1 customer) then
            Tính chi phí Drone Resupply (DJ1) với consolidation
            Tính chi phí Depot Return (DJ2)
            if cost_DJ1 < cost_DJ2 then
                Thực hiện drone resupply (One Rendezvous Model)
            else
                Truck về depot lấy hàng rồi đi giao
            end if
        else if v là Type P (pickup) then
            Truck đi đến P, lấy hàng, đánh dấu cặp pickup
        end if
    end if
    C_max ← max(C_max, T_k*^available)
end for
Áp dụng Merge Strategies nếu có ≥ 2 drone trips

// Assignment Local Search (tùy chọn)
Trích xuất encoding (truck_assign, drone_assign, break_bit) từ lời giải greedy
Chạy Assignment LS (Best-Improvement, 5 operators, tối đa 50 iter)
Nếu LS tìm được lời giải tốt hơn ⇒ cập nhật


return C_max + penalty
```

6.1 Đánh Giá Drone Trip

Cho một drone trip với tập khách hàng $S = \{c_1, c_2, \ldots, c_m\}$:

$$T_{\text{drone_ready}} = \max\left(T_d^{\text{available}}, \max_{c \in S} r_c\right) \tag{3}$$

$$T_{\text{depart}} = T_{\text{drone_ready}} + t_{\text{load}} \tag{4}$$

$$T_{\text{drone_arrive}} = T_{\text{depart}} + \frac{d_E(\text{depot}, c_1)}{v_{\text{drone}}} \times 60 \tag{5}$$

$$T_{\text{truck_arrive}} = T_k^{\text{available}} + \frac{d_M(\text{pos}_k, c_1)}{v_{\text{truck}}} \times 60 \tag{6}$$

$$T_{\text{resupply_start}} = \max(T_{\text{drone_arrive}}, T_{\text{truck_arrive}}) \tag{7}$$

$$T_{\text{resupply_end}} = T_{\text{resupply_start}} + t_{\text{resupply}} \tag{8}$$

$$T_{\text{drone_return}} = T_{\text{resupply_end}} + \frac{d_E(c_1, \text{depot})}{v_{\text{drone}}} \times 60 \tag{9}$$

Ràng buộc endurance: $t_{\text{fly_to}} + t_{\text{wait}} + t_{\text{return}} \leq E_d$

6.2 Consolidation

Nhiều khách hàng Type D có thể được gom vào **một drone trip** nếu thỏa mãn:

- Cùng loại D với $r_i > 0$
- Chênh lệch ready time ≤ 30 phút
- Khoảng cách Euclidean ≤ 10 km
- Số lượng $\leq Q_d$ (drone capacity)

6.3 Merge Strategies

Sau khi decode, chương trình thử **gộp** các drone trip theo 3 chiến lược (tối đa 3 vòng lặp):

1. **Sequential Merge**: Gộp các trip liên tiếp trên cùng truck route
2. **Same-Drone Merge**: Gộp các trip cùng drone (tiết kiệm thời gian bay)
3. **Proximity Merge**: Gộp các trip có khách hàng gần nhau ($d_E < 5$ km)

Chỉ gộp khi improvement > 1 phút và tổng capacity $\leq Q_d$.

6.4 Assignment Local Search (trong Fitness)

Sau khi decode greedy và merge, hàm `decodeAndEvaluate` có thể chạy thêm **Assignment Local Search** để tối ưu hóa việc gán xe/drone cho từng khách hàng mà *không thay đổi thứ tự sequence* π.

Ý tưởng: Giữ nguyên sequence π (thứ tự phục vụ), nhưng thay đổi *ai phục vụ* (truck nào, drone nào, tách/gộp trip) để giảm $C_{\max}$.

Assignment Encoding gồm 3 vector quyết định cho mỗi vị trí i trong sequence:

- **truck_assign**$[i]$: Truck nào phục vụ π_i
- **drone_assign**$[i]$: Drone nào mang hàng (0 = truck tự về depot lấy)
- **break_bit**$[i]$: Bắt đầu drone trip mới (1) hay tiếp tục trip cũ (0)

5 Operators (Best-Improvement — thử tất cả, chọn move tốt nhất):

Quy trình:

1. Trích xuất encoding từ lời giải greedy: `initFromSolution(π, sol)`
2. Lặp tối đa 50 iter:

Bảng 4: Operators trong Assignment Local Search

OP	Tên	Mô tả
1	Flip truck	Đổi $truck_assign[i]$ sang truck khác (kèm DL partner nếu là P)
2	Flip drone	Đổi $drone_assign[i]$ cho Type D: thử $0, 1, \ldots, \lvert\mathcal{D}\rvert$
3	Flip break_bit	Đổi $break_bit[i]$: tách hoặc gộp drone trip tại vị trí i
4	Swap truck	Hoán đổi $truck_assign$ giữa 2 khách hàng khác truck
5	Swap drone	Hoán đổi $drone_assign$ giữa 2 khách hàng Type D

(a) Duyệt tất cả 5 operators trên tất cả vị trí i (và cặp i, j cho OP4, OP5)
(b) Mỗi lần thử: thay đổi encoding → `decodeFromEncoding` → tính cost
(c) Chọn move có cost thấp nhất (best-improvement)
(d) Nếu có cải thiện > 0.01: áp dụng move, tiếp tục
(e) Nếu không: dừng sớm (**break**)

Lưu ý: Khi được bật (`ENABLE_ASSIGNMENT_LS = true`), LS chạy *bên trong mỗi lần gọi fitness*, làm tăng đáng kể chi phí tính toán (phù hợp với $n \leq 20$). Với instance lớn ($n \geq 30$), LS chỉ chạy dạng **post-processing** sau Tabu Search.

7 Genetic Algorithm (GA)

Bảng 5: Tham số GA

Tham số	Giá trị
Population size (N)	200
Max generations	500
Base mutation rate	0.1 (adaptive)
Tournament size	3
Elitism	30% parents giữ lại
Tabu threshold	10% of max generations

7.1 Toán Tử Crossover (Adaptive)

4 toán tử crossover với trọng số tự thích nghi:

1. **Order Crossover (OX)**: Sao chép đoạn từ cha 1, điền phần còn lại từ cha 2 theo thứ tự
2. **Partially Mapped Crossover (PMX)**: Sao chép đoạn + ánh xạ (mapping) để tránh trùng
3. **Cycle Crossover (CX)**: Bảo toàn các cycle giữa hai cha mẹ
4. **Edge Recombination (ERX)**: Bảo toàn các cạnh (adjacency) từ cả hai cha mẹ

Trọng số crossover được cập nhật mỗi $\lfloor \frac{\text{maxGen}}{20} \rfloor$ thế hệ dựa trên success rate:

$$w_i^{\text{new}} = 0.1 + 0.8 \times \frac{\text{success_rate}_i}{\sum_j \text{success_rate}_j} \tag{10}$$

7.2 Toán Tử Mutation (Adaptive)

5 toán tử mutation:

1. **Swap**: Hoán đổi 2 vị trí ngẫu nhiên
2. **Inversion**: Đảo ngược một đoạn con
3. **Scramble**: Xáo trộn một đoạn con
4. **Insertion**: Di chuyển một phần tử đến vị trí khác
5. **Displacement**: Di chuyển một đoạn con đến vị trí khác

Mutation rate tự thích nghi:

$$p_m = \begin{cases} \min(0.5,\ p_m \times 1.2) & \text{nếu không cải thiện} > 5 \text{ thế hệ} \\ \max(0.5 \times p_m^{\text{base}},\ p_m \times 0.9) & \text{nếu đang cải thiện} \end{cases} \tag{11}$$

7.3 Selection

Chọn lọc thế hệ mới theo tỷ lệ:

- 50% offspring tốt nhất (theo fitness tăng dần)
- 20% offspring ngẫu nhiên (từ phần còn lại — duy trì đa dạng)
- 30% parents tốt nhất (elitism)

8 Tabu Search

Tabu Search được kích hoạt khi GA **không cải thiện** sau $\lfloor \frac{\text{maxGen}}{10} \rfloor$ thế hệ liên tiếp.

8.1 Cấu Hình

- Áp dụng cho top 10% cá thể + random 10% cá thể (kèm perturbation)
- Mỗi cá thể chạy Tabu Search với 50 iterations
- Tabu tenure: $\lfloor 0.2 \times n + \text{rand}(0, 10) \rfloor$
- Aspiration criterion: chấp nhận move tabu nếu tốt hơn best-so-far

8.2 6 Loại Move (Adaptive Weights)

Trọng số move được cập nhật mỗi 100 iterations:

$$w_i = (1 - \delta_4) \times w_i + \delta_4 \times \frac{\text{score}_i}{\text{used_count}_i}, \quad \delta_4 = 0.5 \tag{12}$$

8.3 Perturbation (Đa Dạng Hóa)

Trước khi chạy Tabu, các cá thể (trừ top-1) được **perturbation**:

- **Double-Bridge**: Chia sequence thành 4 đoạn, ghép lại theo thứ tự (A, C, B, D)
- **Ruin-Recreate**: Xóa 30% khách hàng ngẫu nhiên, chèn lại vị trí ngẫu nhiên

Bảng 6: Các move trong Tabu Search

ID	Move	Mô tả
0	Swap	Hoán đổi 2 khách hàng
1	Insert	Di chuyển 1 khách hàng đến vị trí khác
2	2-opt	Đảo ngược đoạn $[i, j]$ (giới hạn $j - i \leq 20$)
3	2-opt*	Biến thể 2-opt với đoạn khác (giới hạn $j - i \leq 15$)
4	Or-opt	Di chuyển khối 1–3 phần tử đến vị trí khác
5	Relocate-pair	Di chuyển cặp 2 phần tử liền kề

8.4 Assignment Local Search (Post-Tabu)

Sau Tabu Search, mỗi cá thể chạy **Assignment LS** để tối ưu gán xe/drone:

Algorithm 2 Assignment Local Search (Best-Improvement)

```
Giải mã π để lấy encoding (truck_assign, drone_assign, break_bit)
for iter = 1 to max_iter do
    Tìm move tốt nhất trong 5 operators:
        OP1: Flip truck_assign[i] (thử tất cả trucks ≠ hiện tại)
        OP2: Flip drone_assign[i] cho Type D (thử 0, 1, ..., |D|)
        OP3: Flip break_bit[i] (tách/gộp drone trip)
        OP4: Swap truck_assign giữa 2 khách hàng
        OP5: Swap drone_assign giữa 2 khách hàng Type D
    if Có move cải thiện then
        Áp dụng move, cập nhật encoding
    else
        break {Không còn cải thiện}
    end if
end for
```

Thêm vào đó, **Multi-start**: 3 encoding ngẫu nhiên + 1 greedy, chọn kết quả tốt nhất.

9 Validation (Kiểm Tra Lời Giải)

Chương trình kiểm tra 6 tiêu chí:

1. **Customer Coverage**: Tất cả khách hàng đều được phục vụ
2. **Drone Endurance**: Thời gian bay $\leq E_d = 90$ phút
3. **Timeline Consistency**: Thời gian đến $\geq$ thời gian rời node trước đó
4. **Drone-Truck Sync**: Drone và truck đến trước khi resupply bắt đầu
5. $C_{\max}$ **Verification**: Kiểm tra $C_{\max}$ tính toán khớp với báo cáo
6. **Depot Returns**: Đếm số lần truck quay về depot (chỉ số hiệu quả)

10 Quy Trình Tổng Thể

Algorithm 3 Quy trình tổng thể: GA + Tabu Search

```
Input: Instance file (n khách hàng, tọa độ, ready times)
Output: Lời giải tốt nhất (routes, drone trips, C_max)

// Phase 1: Khởi tạo
Đọc instance, xây dựng ma trận khoảng cách
Khởi tạo N = 200 cá thể (10% Random + 30% Greedy + 30% Sweep + 30% NN)
Đánh giá fitness ban đầu

// Phase 2: Genetic Algorithm
for gen = 0 to 499 do
    Chọn cha mẹ (Tournament Selection, size = 3)
    Tạo offspring bằng Adaptive Crossover (OX/PMX/CX/ERX)
    Mutation adaptive (Swap/Inversion/Scramble/Insertion/Displacement)
    Repair offspring (đảm bảo valid permutation + precedence P→DL)
    Đánh giá offspring: decodeAndEvaluate
    Selection: 50% best offspring + 20% random offspring + 30% best parents
    Cập nhật best solution

    if Không cải thiện ≥ ⌊maxGen/10⌋ thế hệ then
        // Phase 3: Tabu Search Intensification
        Chạy Tabu Search (50 iter) cho top 10% + random 10% (kèm perturbation)
        Chạy Assignment LS (multi-start) cho mỗi cá thể sau Tabu
        Cập nhật quần thể
    end if
    Cập nhật trọng số adaptive operators
end for

// Phase 4: Output
In lời giải chi tiết (truck routes, drone trips, timeline)
Kiểm tra tính hợp lệ (Validation)
Báo cáo C_max cuối cùng
```

11 Instances

Các instance được đặt tên theo format: `U_n_β_Num_i.txt`, trong đó:

- `U`: Phân bố khách hàng đều (Uniform)
- `n`: Số khách hàng ($n \in \{10, 30, 50, 100\}$)
- β: Hệ số spread của ready time ($\beta \in \{0.5, 1.0, 1.5, 2.0, 2.5, 3.0\}$)
- `i`: Số thứ tự instance ($i \in \{1, \ldots, 10\}$)

Mỗi instance chứa danh sách node với: ID, tọa độ (x, y), loại (P/DL/D), ready time r_i, pair ID.

Depot mặc định tại $(10, 10)$, có thể thay đổi qua tham số dòng lệnh `-depot x,y`.

12 Sử Dụng Chương Trình

```
# Biên dịch
```

```
g++ -O2 -std=c++17 -o main_ga_tabu.exe src/*.cpp

# Chạy với depot mặc định (10, 10)
./main_ga_tabu.exe "Instances 2/U_30_0.5_Num_1.txt"

# Chạy với depot tùy chỉnh (-10, 10)
./main_ga_tabu.exe "Instances 2/U_30_0.5_Num_1.txt" --depot -10,10
```